

Sorting and Patterns - Day 4

Heapsort and fast/slow pointers

Priyanka's Platinum study plan | Thursday, September 10, 2026 | 120 minutes | C11 and Python 3

Today's question: what structure can we encode without allocating a second data structure? An array can encode a heap; two linked-list pointers can reveal position and cycles. Prerequisites: array bounds, Day 3's recursive reasoning, and basic C structs/pointers. Start in C today, then translate to Python. Node definitions are supplied in the appendix; reuse them rather than spending the session on list input/output.

Your two-hour session

Minutes	Task	Deliverable
0-15	Draw the array-backed heap	Parent/child formulas
15-50	Lab A: build heap and heapsort, both languages	Iterative sift-down
50-70	Heap traces and runtime problems	Linear build proof
70-85	Derive middle and cycle detection	Pointer-position trace
85-110	Lab B: implement middle/cycle detection in both languages	Boundary tests
110-120	Lab C: trace cycle entry and repair reference	Reset-pointer proof

Lab C is a guided trace and reference-code exercise within the final ten minutes; a second full independent implementation is not added to today's workload. Attempt each task before opening its solution section.

Lab A - From repeated maximum selection to a heap

Repeatedly scanning for the maximum costs $\Theta(n^2)$, as in selection sort. Instead maintain a structure whose maximum is always at index 0. A max heap has two properties: its shape is a complete binary tree, and every parent key is at least each child key. It is NOT a fully sorted array; siblings have no required relative order.

For zero-based indices, left child is $2r+1$, right child $2r+2$, and parent of $r>0$ is $\text{floor}((r-1)/2)$. The heap occupies $[0, \text{size})$. A node has a child precisely when $r < \text{size}/2$. Derive this before using it: for $n=5$, nodes 0 and 1 are parents and nodes 2,3,4 are leaves. For $n=6$, node 2 has only its left child.

Sift-down invariant

Assume both child subtrees are heaps and only root may violate heap order. Choose the larger EXISTING child. If $\text{root} \geq$ that child, both parent-child edges are valid and repair is complete. Otherwise swap root with that child and continue there. The larger child moved up dominates both child roots; any remaining violation moves downward along one path. Each step increases depth, so it terminates in $O(\log \text{size})$ worst time.

Implementation details: test whether the right child exists before reading it. Use the active heap size, not the original array length. In C, checking $\text{root} < \text{size}/2$ before computing $2*\text{root}+1$ also keeps child-index arithmetic within the active range. Stop on equal values; swapping equals is unnecessary.

Build then extract

Build bottom-up from the last parent to the root. Before processing a parent, its children are leaves or already-repaired heaps. This proves the sift-down precondition by reverse induction. In C, avoid a loop that decrements unsigned root below zero; start at $n/2$ and process $\text{start}-1$ while $\text{start} > 0$.

For heapsort, swap the maximum root into the final active position, shrink the heap by one, and sift the replacement root down. Outer invariant: the active prefix is a max heap, the suffix is sorted, and every suffix value is at least every active value. Never include the extracted suffix in sift-down.

Implement `heap_sort(a)` in Python and `heap_sort(a,n)` in C. Both mutate the input, use iterative sift-down and allocate no auxiliary array. Test `[]`, `[1]`, `[2,1]`, `[1,2,3,4,5,6]`, `[4,10,3,5,1]`, and `[2,2,2,2]`.

Heap analysis workshop

1. Build a heap from `[4,10,3,5,1]`. After each extraction, record both the active prefix and fixed suffix.
2. Why is n times the maximum sift cost a valid $O(n \log n)$ upper bound for building a heap but not a tight bound?
3. At most about $n/2^{(h+1)}$ nodes have height h in a full binary tree. Sum the work proportional to $n * \sum(h/2^{(h+1)})$. Explain why leaves contribute almost no repair work and the total is $O(n)$, also for incomplete final levels up to constant factors.
4. Compare bottom-up construction with n successive heap insertions, which can take $\Theta(n \log n)$ in the worst case.
5. This sift-down stops early on equal keys. What happens to the BEST-case sorting time on an all-equal array? Distinguish this from its worst-case guarantee.

Lab B - One pointer samples twice as fast

Represent a singly linked node by a value and a next reference. Compare node identity/address, not stored values: two distinct nodes may hold the same integer. Functions must not mutate links.

Middle contract: on an acyclic list, return the middle node; for even length, return the second middle; for an empty list, return `null/None`. After t iterations, slow has taken t links and fast $2t$ links, provided both fast moves exist. Advance while fast and fast.next exist. Trace lengths 0 through 6. A two-node list returns its second node.

Cycle contract: return whether repeatedly following next ever revisits a node. The naive method stores every visited address in a set, using $O(n)$ memory. Can the two speeds detect repetition with $O(1)$ state? Advance both pointers, then test whether they meet. Checking their equality before the first move would report a cycle in every nonempty list.

Once both are in a cycle of length λ , fast gains one position modulo λ per iteration. Therefore they meet within at most λ further iterations. If there is no cycle, fast reaches null and the guarded loop terminates.

Runtime is $O(n)$ over the number of distinct reachable nodes; workspace $O(1)$.

Tests: null head, a singleton ending in null, a self-cycle, two nodes ending in null, a two-node cycle, equal-valued distinct nodes, and a cycle beginning after a prefix. Do not call middle on a cyclic list; acyclicity is part of its contract.

For C tests, a stack array of Node objects avoids allocation cleanup: link each element to the next, then optionally point the tail into the array. For heap-allocated test nodes, disconnect the tail's cycle edge before freeing the list, or free using a separately retained allocation list. Never traverse a cycle to null expecting cleanup to terminate.

Lab C - Where does the cycle begin?

Trace `A->B->C->D->E->C`. First locate the slow/fast meeting. Then reset a third pointer to head and move it and the meeting pointer one step at a time until they meet. Predict the returned node before reading the reference.

Let μ be the number of links before the entry and λ the cycle length. If slow has taken t steps at a meeting, fast has taken $2t$; their difference t is a multiple of λ . Slow's offset from entry is $(t-\mu)$ modulo λ . After μ further steps its offset is t modulo $\lambda=0$, exactly when a head pointer reaches the entry. Explain why comparing VALUES rather than addresses invalidates this argument.

Exit: give the heap-build sum, distinguish heap shape from heap order, and show the two pointer guards needed before fast advances twice. Submit both languages' heapsort/middle/cycle code and the cycle-entry trace. Read the supplied entry implementation and mark the reset, equality check, and two one-step advances.

Solution appendix

Heap trace and cost checks

[4,10,3,5,1] becomes [10,5,3,4,1] after construction. Successive repaired active prefixes and suffixes are [5,4,3,1] / [10], [4,1,3] / [5,10], [3,1] / [4,5,10], and [1] / [3,4,5,10]. Final order is [1,3,4,5,10]. Swapping only with the left child fails, for example, at root 1 with children 2 and 3.

The height-weighted sum converges: $\sum(h/2^{(h+1)})$ for $h \geq 0$ equals 1. Thus bottom-up build is $O(n)$, and the traversal itself supplies an $\Omega(n)$ bound, so total build time is $\Theta(n)$. Sorting has $O(n \log n)$ upper time and $\Theta(n \log n)$ worst time; repeated maximum removal is much cheaper than scanning the entire prefix each time. Under uniformly random distinct permutations its average time is $\Theta(n \log n)$.

For THIS implementation on all-equal arrays, every sift stops after its first comparison, while construction and extraction each visit $O(n)$ nodes/roots. Therefore best time over arrays allowing duplicates is $\Theta(n)$, not an unconditional $\Theta(n \log n)$. With distinct keys, the conventional heapsort analysis gives $\Theta(n \log n)$ best time. Always state the input assumptions and early-stop behavior. Iterative sift-down keeps auxiliary space $O(1)$; heapsort is not stable.

Linked-list answers

For lengths 0,1,2,3,4,5,6, the returned zero-based middle indices are none,0,1,1,2,2,3. A self-cycle is detected after one move. For $A \rightarrow B \rightarrow C \rightarrow D \rightarrow E \rightarrow C$: slow/fast positions after moves are B/C, C/E, D/D. Reset seeker to A, then move seeker/slow through B/E to C/C. The cycle entry is C.

Cycle entry takes $O(\mu + \lambda)$ time and $O(1)$ workspace, including detection and the reset phase. For no cycle it returns null/None. Node values play no role in either detection or entry.

Progressive hints

Heap: first mark which indices are leaves, then repair parents in reverse order. Middle: draw fast's landing position, including null. Cycle: compare after moving. Entry: reason modulo λ and keep μ as a distance, not a node value.

Python 3 reference functions

PY

```
def sift_down(a, root, size):
    while root < size // 2:
        child = 2 * root + 1
        if child + 1 < size and a[child + 1] > a[child]:
            child += 1
        if a[root] >= a[child]:
            break
        a[root], a[child] = a[child], a[root]
        root = child
```

PY

```
def heap_sort(a):
    for root in range(len(a) // 2 - 1, -1, -1):
        sift_down(a, root, len(a))
    for end in range(len(a) - 1, 0, -1):
        a[0], a[end] = a[end], a[0]
        sift_down(a, 0, end)
```

PY

```
class Node:
    def __init__(self, value, next_node=None):
        self.value = value
        self.next = next_node
```

PY

```
def middle(head):
    # Contract: acyclic list; returns second middle for even length.
    slow = fast = head
    while fast is not None and fast.next is not None:
        slow = slow.next
        fast = fast.next.next
    return slow
```

PY

```
def has_cycle(head):
    slow = fast = head
    while fast is not None and fast.next is not None:
        slow = slow.next
        fast = fast.next.next
        if slow is fast:
            return True
    return False
```

PY

```
def cycle_entry(head):
    slow = fast = head
    while fast is not None and fast.next is not None:
        slow = slow.next
        fast = fast.next.next
        if slow is fast:
            seeker = head
            while seeker is not slow:
                seeker = seeker.next
                slow = slow.next
            return seeker
    return None
```

C11 reference functions

C

```
#include <stdbool.h>
#include <stddef.h>
```

C

```
void sift_down(int *a, size_t root, size_t size) {
    while (root < size / 2) {
        size_t child = 2 * root + 1;
        if (child + 1 < size && a[child + 1] > a[child]) ++child;
        if (a[root] >= a[child]) break;
        int t = a[root]; a[root] = a[child]; a[child] = t;
        root = child;
    }
}
```

C

```
void heap_sort(int *a, size_t n) {
    for (size_t start = n / 2; start > 0; --start)
        sift_down(a, start - 1, n);
    for (size_t size = n; size > 1; --size) {
        int t = a[0]; a[0] = a[size - 1]; a[size - 1] = t;
        sift_down(a, 0, size - 1);
    }
}
```

C

```
typedef struct Node { int value; struct Node *next; } Node;
```

C

```
Node *middle(Node *head) {  
    /* Contract: acyclic list. */  
    Node *slow = head, *fast = head;  
    while (fast && fast->next) {  
        slow = slow->next;  
        fast = fast->next->next;  
    }  
    return slow;  
}
```

C

```
bool has_cycle(Node *head) {  
    Node *slow = head, *fast = head;  
    while (fast && fast->next) {  
        slow = slow->next;  
        fast = fast->next->next;  
        if (slow == fast) return true;  
    }  
    return false;  
}
```

C

```
Node *cycle_entry(Node *head) {  
    Node *slow = head, *fast = head;  
    while (fast && fast->next) {  
        slow = slow->next;  
        fast = fast->next->next;  
        if (slow == fast) {  
            Node *seeker = head;  
            while (seeker != slow) {  
                seeker = seeker->next;  
                slow = slow->next;  
            }  
            return seeker;  
        }  
    }  
    return NULL;  
}
```